Lab 03 — Chaotic Text Generator: HMM vs trigram test bench

App: **https://hmm-text-generator.streamlit.app** · Code: **https://github.com/hichamouaouche/hmm-text-generator** · All numbers: 500 generated sentences, seed 42.

1. Problem and solution

A text generator has no reference answer, so its quality cannot be judged from a few examples; it must be measured on hundreds of sentences with automatic metrics. We estimate a supervised POS-HMM (start, transition and emission tables counted on 960 training sentences, 6 tags) and generate by sampling a tag sequence and then one word per tag, with a temperature (weights count$^{1/T}$). We compare plain states, role-split states (S / O / P) and a trigram baseline, and tag sentences with log-space Viterbi (add-k, k = 0.01). A Streamlit test bench exposes generation, four metrics, a Viterbi tagger and a CSV export.

2. Metrics at T = 1.0

Model	Valid tag pattern	Plausible subject-verb	Novel sentences	Distinct-2
HMM, plain states	16.2%	14.8%	99.8%	0.198
HMM, role-split states	**100%**	60.6%	99.4%	0.148
Trigram	57.0%	**76.0%**	91.2%	0.105

Why the plain HMM scores low. With first-order transitions the next tag depends only on the current tag. After a NOUN the model draws VERB (0.39), PREP (0.22) or PUNCT (0.39) whatever the role of the noun, so it ends a sentence right after the subject (*“the quiet school .”*) or adds a second verb after the object (*“the boys visit the rice sell the young farmers .”*).
Why role-split reaches 100%. NOUN_S, NOUN_O and NOUN_P are different states: NOUN_S is always followed by VERB, NOUN_O by PREP or PUNCT, NOUN_P by PUNCT. The state now carries the position in the sentence, so every sampled path follows one of the 16 training patterns. (Tagging accuracy is 100% for both, including *watch* 37/37.)

3. Choice of temperature

- **Writing toy: role-split HMM at T = 1.5.** Structure stays at 100% at every temperature while diversity rises (distinct-2 0.107 → 0.161 from T = 0.5 to 1.5), and odd sentences are acceptable or even fun in a toy.
- **Autocomplete: trigram at T = 0.5.** A suggestion must be safe and predictable: at T = 0.5 the trigram has its best valid-pattern (63.6% vs 44.2% at T = 2.5) and subject-verb plausibility (79.2%); its repetitiveness and copies are not a problem for completion.

4. One good and one bad sentence

- **Good** (role-split, T = 1.0): *“the children eat the bread at the park .”* — valid pattern, plausible pair, novel.
- **Bad** (role-split, T = 1.0): *“the foxes lose by the school .”* — failure type **meaning**: the pattern DET NOUN VERB PREP DET NOUN PUNCT is valid, but the pair (foxes, lose) never occurs. Cause: the verb is emitted from the VERB state only, independently of the subject noun, so states give grammar but not meaning.
- Other failure types seen: **structure** — *“the happy market .”* (plain HMM, invalid pattern DET ADJ NOUN PUNCT); **memorisation** — *“the girls leave the park .”* (trigram, T = 0.5, copied verbatim from training: not creative).

5. What state splitting changed, and what it did not

Splitting NOUN, ADJ and DET by role gave the first-order model the memory it lacked: the valid tag pattern rose from 16.2% to 100% at every temperature, the model still generates new sentences (99.4% novel) and Viterbi tagging stays at 100%. It did not give the model meaning: words are still emitted independently given the state, so subject-verb plausibility is only 60.6%, below the trigram (76.0%), which sees the actual previous words. Diversity also drops (distinct-2 0.198 → 0.148) because the paths are more constrained. This is a negative result worth reporting: 100% structure is not 100% quality.

6. Real-data test (UD English EWT, 12,544 train / 2,077 test sentences, 17 tags)

Model	Accuracy	Known words	Unknown words	Whole sentence correct
Synthetic corpus, HMM + Viterbi	100%	100%	—	100%
EWT, most-frequent tag (baseline)	86.2%	90.8%	28.6%	31.9%
EWT, HMM + Viterbi	88.1%	92.9%	28.2%	40.4%
EWT, HMM + Viterbi + suffix model (extension)	91.7%	93.6%	67.7%	49.4%

The synthetic grammar has 6 tags, a small closed vocabulary, 16 tag sequences and an ambiguity (*watch*) that the previous tag always resolves, so 100% is expected and says nothing about real English. On real web text the same code gains only 1.9 points over the most-frequent-tag baseline and gets 40.4% of whole sentences right. 7.5% of test tokens are unknown and are tagged at only 28.2%: the smoothed emission k / (count(tag) + k·V) is largest for rare tags, so unknown words drift to X and INTJ (PROPN→X 195, PROPN→INTJ 137). With 10,359 distinct tag sequences for 12,544 sentences, the valid-tag-pattern metric is meaningless on real text. **Extension:** estimating P(tag | suffix) on rare training words for unseen words (TnT-style, parameters fixed before testing) raises unknown-word accuracy to 67.7% and overall accuracy to 91.7%; the remaining errors are mostly PROPN/NOUN, which lower-casing makes hard to separate.